1 Deep Reinforcement Learning for Music Recommendation

1.1 Problem Formulation

We model music recommendation as a **sequential decision-making problem** using Reinforcement Learning (RL). Unlike traditional recommender systems that optimize for immediate reward, our objective is to maximize **long-term user satisfaction** by accounting for temporal dependencies in user behavior.

The problem is formulated as a **Markov Decision Process (MDP)** defined by the tuple:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma)$$

where:

- $\mathcal{S}$ is a continuous state space representing the user context
- $\mathcal{A}$ is a discrete action space corresponding to available songs (or continuous embeddings of songs)
- $\mathcal{R}$ is a continuous reward function derived from user feedback
- $\mathcal{P}$ is the state transition dynamics
- $\gamma \in (0, 1)$ is a discount factor

1.2 State Representation

At time step t, the state $s_t \in \mathbb{R}^d$ encodes the user's listening context and history:

$$s_t = \left[\bar{x}_t, x_{t-1}, r_{t-1}, n_t, h_t\right]$$

where:

- $\bar{x}_t$ is the exponentially weighted average of audio features of previously liked songs
- x_{t-1} is the audio feature vector of the previously recommended song
- r_{t-1} is the reward obtained at the previous step
- n_t is a novelty score modeling repeated exposure effects
- h_t is a latent embedding of the session history obtained via a recurrent encoder

1.3 Action Space

The action space can be treated in two ways:

- **Discrete**: recommend a song from a candidate pool
- **Continuous embedding**: map songs to a continuous feature space; the policy outputs a vector, and we select the nearest song

1.4 Reward Function

User feedback is mapped to a continuous scalar reward:

$$r_t \in \{-5, -2, 0, 2, 5\}$$

Optionally, the reward can be refined using listening duration:

$$r_t = \alpha \cdot r_t^{\text{explicit}} + (1 - \alpha) \cdot \frac{\text{ms\_played}_t}{\text{track\_duration}_t}, \quad \alpha \in [0, 1]$$

1.5 Soft Actor-Critic (SAC)

SAC is an off-policy algorithm that learns a stochastic policy $\pi_\phi(a|s)$ and two Q-functions $Q_{\theta_i}(s, a)$ with entropy regularization. The policy objective is:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\pi}\Big[Q(s_t, a_t) - \alpha \log \pi_\phi(a_t|s_t)\Big]$$

The Q-functions are updated with the Bellman backup:

$$y_t = r_t + \gamma \mathbb{E}_{a_{t+1}\sim\pi}\Big[\min_{i=1,2} Q_{\theta_i^-}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\phi(a_{t+1}|s_{t+1})\Big]$$

$$\mathcal{L}(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(Q_{\theta_i}(s, a) - y_t\big)^2\Big]$$

1.6 Proximal Policy Optimization (PPO)

PPO updates the policy by maximizing the clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\phi) = \mathbb{E}_t\Big[\min\big(r_t(\phi)\hat{A}_t, \text{clip}(r_t(\phi), 1 - \epsilon, 1 + \epsilon)\hat{A}_t\big)\Big]$$

where

$$r_t(\phi) = \frac{\pi_\phi(a_t|s_t)}{\pi_{\phi_{\text{old}}}(a_t|s_t)}$$

and $\hat{A}_t$ is the advantage estimate, ϵ is the clipping parameter.

1.7 Offline RL Considerations

Both SAC and PPO can be adapted to offline RL by:

- Restricting the action space to observed actions in the dataset
- Normalizing or clipping rewards
- Conservative policy updates to avoid distributional shift

1.8 Neural Network Architecture

For both SAC and PPO, we use:

- A fully connected state encoder
- Non-linear activations (ReLU)
- For SAC: outputs mean and std for action distribution
- For PPO: outputs action probabilities (discrete) or continuous action parameters
- Critic networks estimating $Q(s, a)$ or value function $V(s)$

1.9 Discussion

Using SAC or PPO allows us to handle continuous state representations, long-term engagement optimization, and stochastic policies that improve exploration. SAC is preferred for offline learning due to its off-policy nature, while PPO can be used with careful batch management.